

Agentic AI for Code Generation

Seminar Report

Student: "Thua Duc Nguyen"✉ and Supervisor: "Derui Zhu"✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ thuaduc.nguyen@tum.de

✉ derui.zhu@tum.de

April 1, 2025

Abstract —

1 Introduction

Motivation. The rise of “vibe coding” and AI-assisted software development: how natural-language-driven programming is reshaping the developer workflow.

Background and Related Surveys. Positioning relative to prior surveys on LLM-based code generation, automated program repair, and AI agent architectures. What this report adds: a pipeline-centric synthesis across four active research fronts.

Scope and Contribution. Define the scope (autonomous code-generation agents, 2023–2025 literature), state the contribution (unified pipeline framing, cross-topic synthesis), and outline the report structure.

2 The Vibe Coding Pipeline

Present the vibe coding pipeline as a unifying conceptual framework and compare it to classical SE process models (waterfall, agile inner loops).

2.1 Pipeline Stages

Formal definition of each stage: Intent Capture → Planning → Code Generation → Execution → Refinement. Five stages forming a loop: execution feedback drives refinement, which may trigger re-planning or re-generation. What inputs and outputs connect adjacent stages.

2.2 Mapping to Survey Topics

Explicit mapping: how each of the four subsequent sections corresponds to one or more pipeline stages. §3 covers the full pipeline as an integrated system;

§4 zooms into planning; §5 covers multi-agent approaches to generation and validation; §6 covers the execution-feedback-refinement loop (including fault-specific self-repair). Cross-cutting concerns (evaluation, context management) are consolidated in the Discussion.

3 End-to-End Autonomous Software Engineering Agents

Before examining individual pipeline stages, we survey systems that orchestrate the *entire* vibe coding pipeline, accepting a natural-language issue or specification and producing a tested code change with minimal human intervention. These end-to-end agents provide the holistic context in which planning (§4), multi-agent collaboration (§5), and feedback-driven refinement (§6) operate.

We begin by identifying the architectural patterns that underlie most current systems (§3.1). We then survey five representative agents that span the design space: from fully autonomous to deliberately non-agentic (§3.2). Finally, we examine how these systems are evaluated, focusing on SWE-bench and its variants (§3.3). Figure 1 provides a taxonomy of representative systems along four key dimensions: orchestration approach, action interface design, context management strategy, and evaluation methodology.

3.1 Architecture and Orchestration

Despite surface-level diversity, autonomous SE agents converge on a small number of architectural patterns. As illustrated in the *Orchestration*, *Action Interface*, and *Context Management* branches of Figure 1, these patterns determine how agents perceive their environment, what actions they can take, and how they manage repository-scale context.

The observe–think–act loop. The dominant paradigm is a ReAct-style [15] loop in which the agent

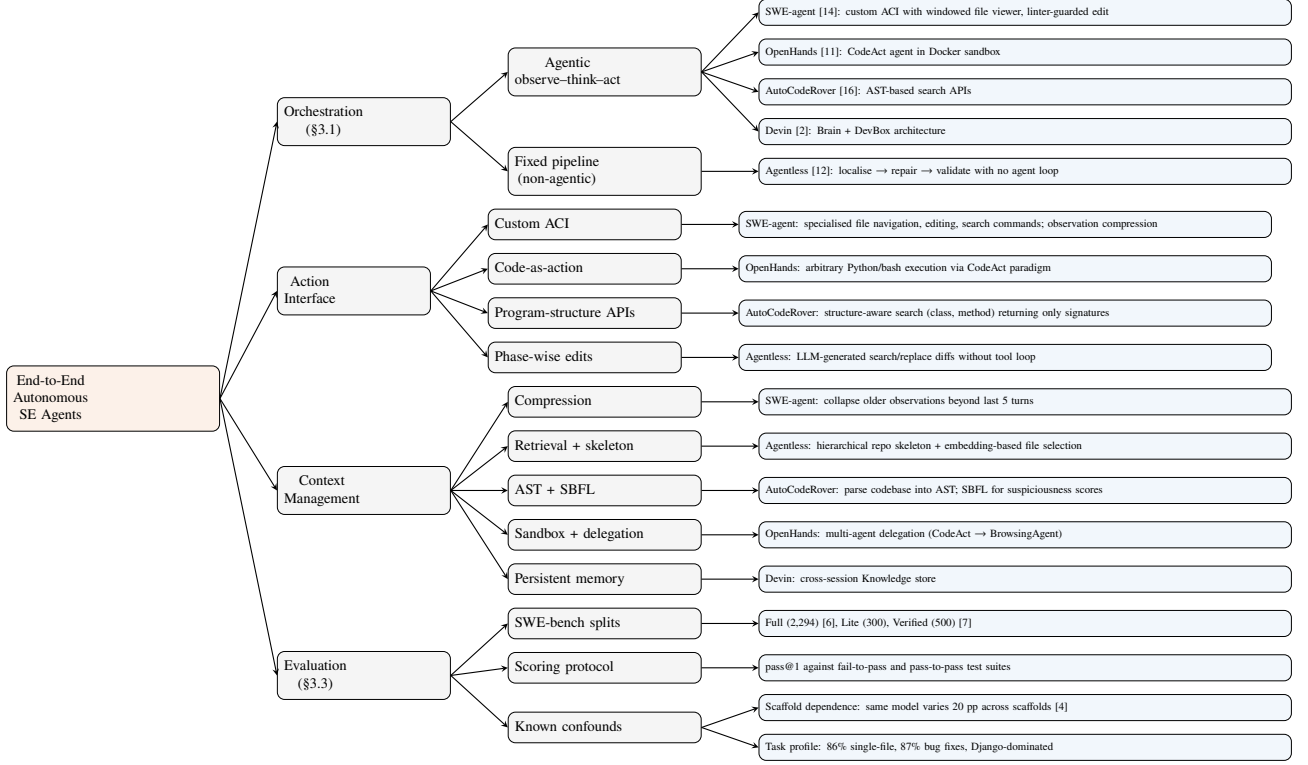


Figure 1 Taxonomy of end-to-end autonomous SE agents. Systems are classified along orchestration approach (agentic loop vs. fixed pipeline), action interface design (custom ACI, code-as-action, structure-aware APIs), context management strategy (compression, retrieval, AST, delegation, memory), and evaluation methodology (benchmark splits, scoring, known confounds). Representative systems are shown as leaf exemplars.

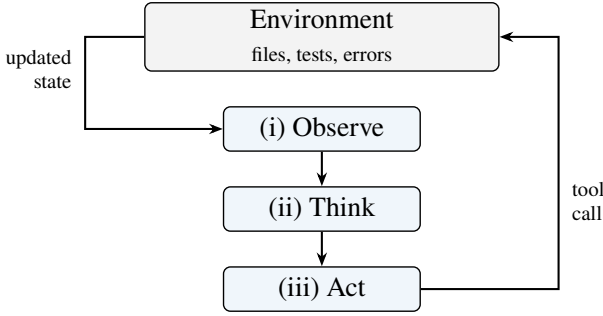


Figure 2 ReAct-style observe-think-act loop. The agent reads environment state, reasons about the next step, executes a tool call, and repeats.

alternates between (i) observing the environment (file contents, test output, error traces), (ii) reasoning about what to do next (chain-of-thought or structured plan), and (iii) executing an action via a tool call. SWE-agent [14] and OpenHands [11] both instantiate this loop, differing mainly in the *action space* exposed to the model (Figure 2).

Agent-Computer Interface (ACI). Yang et al. [14] introduce the ACI concept, arguing that LLMs are a “new category of end users” that need purpose-built in-

terfaces rather than raw shell access. SWE-agent’s ACI includes a windowed file viewer (100 lines), a linter-guarded `edit` command, summarised search results, and compressed observation history. Ablations show that each ACI component contributes measurably: removing the linter drops performance by 3 percentage points; switching to iterative search costs 6 pp; and showing full file content instead of a 100-line window costs 5.3 pp on SWE-bench Lite.

Code-as-action. OpenHands adopts the CodeAct paradigm [10], consolidating all agent actions into executable Python and bash code rather than a fixed tool API. Each turn may run arbitrary IPython cells, bash commands, or browser interactions inside a Docker-sandboxed environment. This design maximises flexibility at the cost of a larger action space that the model must navigate.

Context management. Repository-level tasks easily exceed LLM context windows. Agents employ several mitigation strategies: SWE-agent collapses older observations beyond the last 5 turns; AutoCodeRover [16] parses the codebase into an

Table 1 Comparison of end-to-end autonomous SE agents. Architectural patterns: **OTA** = observe–think–act loop, **ACI** = custom Agent–Computer Interface, **CaA** = code-as-action, **Pipe** = fixed multi-phase pipeline (no agent loop). SWE-bench scores are single-pass (pass@1) unless noted; **Lite** = SWE-bench Lite (300 instances), **Full** = SWE-bench Full (2,294 instances). Cost is average USD per task instance.

System	Origin	Architecture	Context Strategy	Lite	Full	Cost
SWE-agent	Princeton	OTA + ACI	Observation compression	18.0%	12.5%	\$1.21
Agentless	UIUC	Pipe	Hierarchical skeleton + embeddings	32.0%	–	\$0.70
AutoCodeRover	NUS	OTA + AST search	AST-based APIs + SBFL	19% [†]	12.4%	\$0.43
OpenHands	Multi-inst.	OTA + CaA	Docker sandbox + delegation	26.0%	–	–
Devin	Cognition	OTA + Brain/DevBox	Persistent knowledge store	– [‡]	–	propr.

[†]26% with 3 retries (pass@3). [‡]Proprietary; see text for Verified estimates.

AST and exposes structure-aware search APIs (`search_class`, `search_method_in_class`) that return only signatures, not full definitions; Agentless [12] constructs a hierarchical repository skeleton and uses embedding-based retrieval to select candidate files before prompting the LLM.

3.2 Representative Systems

We survey five prominent systems that span the design space. Table 1 provides a comparative overview.

SWE-agent [14]. Developed at Princeton, SWE-agent wraps a GPT-4 Turbo backbone in a custom ACI with specialised file navigation, editing, and search commands. It achieved 12.47% on the full SWE-bench test set (2,294 tasks) and 18.00% on SWE-bench Lite when first published. The agent follows a typical trajectory: reproduce the issue, localise the faulty code via `find_file/search_dir`, then enter an edit–execute loop. The median successful run completes in 12 steps at \$1.21 API cost.

Agentless [12]. From UIUC, Agentless challenges the need for agentic autonomy. It replaces the interactive loop with a fixed three-phase pipeline: *localise* → *repair* → *validate*. Localisation uses a hierarchical narrowing strategy (files → classes/functions → edit locations) combining LLM prompting with embedding retrieval. Repair generates up to 40 candidate patches in a search/replace diff format. Validation selects the best patch via regression testing, LLM-generated reproduction tests, and majority voting over AST-normalised candidates. Agentless resolved 32.00% of SWE-bench Lite at an average cost of only \$0.70 per problem, making it the top open-source system at the time and establishing a strong non-agentic baseline. OpenAI later

adopted Agentless as its default scaffold for showcasing GPT-4o and o1 on SWE-bench Verified.

AutoCodeRover [16]. From NUS, AutoCodeRover takes a “software-engineering-oriented” approach, operating on program representations rather than raw files. It parses the codebase into an AST and provides the LLM with structure-aware search APIs. A stratified context retrieval process iteratively expands the agent’s understanding, starting from issue keywords and refining through multiple search rounds until sufficient context is gathered. When a test suite is available, Spectrum-Based Fault Localisation (SBFL) augments the search with suspiciousness scores. AutoCodeRover achieved 19% on SWE-bench Lite in a single run and 26% with three retries, at a cost of \$0.43–\$1.30 per task.

OpenHands [11]. Formerly OpenDevin, OpenHands is an open-source platform (ICLR 2025, 32K GitHub stars, 188 contributors) built around the CodeAct agent. Each session runs inside a Docker container with a bash shell, IPython server, and Chromium browser. The platform supports multi-agent delegation—a generalist CodeAct agent can offload web browsing to a specialised BrowsingAgent. On SWE-bench Lite, CodeAct v1.8 with Claude 3.5 Sonnet achieved 26.0%. On SWE-bench Verified, a later submission reaches 60.6% with a single rollout and 66.4% when five trajectories are reranked by a dedicated critic model [8].

Devin (Cognition) [2]. Devin is a proprietary cloud-hosted agent positioned as an “autonomous AI software engineer.” Its architecture separates a *Brain* (reasoning layer) from a *DevBox* (execution VM with shell, IDE, and browser). A persistent Knowledge store provides cross-session memory. Cognition re-

Table 2 SWE-bench dataset splits.

Split	Size	Selection criterion
Full	2,294	All filtered PRs
Lite	300	Subset selected for tractability
Verified	500	Human-validated (3 annotators per instance)

ports strong full-benchmark results, but independent leaderboard analysis finds 68.3% on SWE-bench versus 48.2% on SWE-bench Verified (a 29% relative drop), suggesting possible overfitting to easier unfiltered instances [3].

Taken together, these five systems expose a recurring trade-off between agentic flexibility and pipeline efficiency. Agentless shows that a fixed localise–repair–validate pipeline can outperform interactive agents on SWE-bench Lite at lower cost (\$0.70 vs. \$1.21), challenging the assumption that autonomy is necessary for strong results. Among agentic designs, interface and context strategy matter as much as the backbone LLM: SWE-agent’s ACI ablations, OpenHands’s CodeAct action space, and AutoCodeRover’s AST search APIs each demonstrate that how an agent reads and edits code can shift resolve rates by several percentage points at comparable model capacity.

3.3 Full-Pipeline Evaluation

The *Evaluation* branch of Figure 1 summarizes the primary benchmark splits, scoring protocols, and known confounds that shape current performance reports.

SWE-bench: dataset construction. The primary evaluation instrument for end-to-end agents is SWE-bench [6]. It was constructed by scraping merged pull requests from 12 popular open-source Python repositories (including Django, scikit-learn, sympy, matplotlib, and Flask). Each PR was filtered to retain only those that modify the project’s test suite, yielding 2,294 task instances in the **Full** split. For each instance the benchmark provides: (i) the repository snapshot *before* the PR was merged, (ii) the issue description (title + body), (iii) a gold patch (the human-written fix), and (iv) a set of *fail-to-pass* tests (which fail before the fix and should pass after) and *pass-to-pass* tests (which must continue to pass). An agent receives only items (i) and (ii); its generated patch is applied and scored against both test sets.

Splits and filtering. Three subsets are commonly used (Table 2):

SWE-bench Lite selects 300 instances for faster iteration but was found to contain quality issues: Xia et al. [12] report that 4.3% of instances contain the exact gold patch in the issue text, 9.7% provide complete solution steps in natural language, and 10% lack sufficient information to be solvable. **SWE-bench Verified** [7] addresses these concerns: three independent human annotators assessed each Full instance for problem clarity, test correctness, and overall solvability, retaining 500 high-quality instances. The Epoch AI analysis [4] estimates a remaining error rate of 5–10%. Verified has become the de facto standard for leaderboard comparisons.

Task complexity profile. Although SWE-bench tests real-world issues, most tasks are relatively small [5, 4]. In the Verified split, human annotators estimated that 39% of tasks are trivial (<15 min fix), 52% are small (15 min–1 h), 8% are substantial (1–4 h), and only 3 tasks were rated >4 h. These estimates correlate with patch size: easy tasks average 5 changed lines of code (LOC) and 1.0 files; medium tasks average 14 LOC and 1.3 files; hard tasks average 56 LOC and 2.0 files. Crucially, 86% of instances are single-file changes, and 87% are classified as bug fixes rather than feature requests or refactorings. The dominance of Django (nearly 50% of instances) and four other repositories (>80% combined) further limits diversity.

Scaffold dependence. An important confound is that *performance reflects scaffold design as much as model capability* [4]. The same model can vary by 20 pp depending on the scaffold: for example, Claude 3.7 Sonnet scores 62.3% with one scaffold and 70.2% with another; GPT-4o scores 23% under SWE-agent but 33.2% under Agentless. This makes cross-system comparisons difficult unless the scaffold is controlled, motivating standardised evaluation harnesses such as Live-SWE-agent [13].

Leaderboard trajectory. Figure 3 charts the rapid progress on SWE-bench from late 2023 through mid-2026. In October 2023, the best RAG baselines resolved 1.96% of Full and 0.33% of Lite [6]. SWE-agent’s agentic loop lifted Full to 12.5% and Lite to 18.0% in early 2024. On the Full set, AutoCodeRover reached 18.0% (pass@3) and Devin 13.9% around the same period. The scaffolding race then accelerated both tracks: by late 2024, Agentless 1.5 with Claude 3.5 Sonnet reached 40.7% on Lite; by mid-2025 the best Lite score stood at 60.3%. On the Full

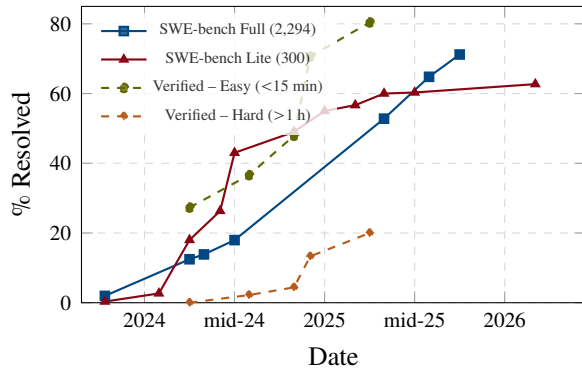


Figure 3 SWE-bench progress over time. Solid lines: best-in-class resolve rate on the Full set (blue, 2,294 tasks) and Lite (red, 300 tasks). Dashed lines: performance on easy (<15 min, green) vs. hard (>1 h, orange) subsets of Verified, illustrating the widening difficulty gap [5]. Data compiled from leaderboard submissions [9, 3, 1].

2,294-instance set, fewer teams report results post-2024, but dataku’s independent analysis [3] records Claude Opus 4 at 71.2% and Devin at 68.3%. On Lite, the frontier reached ~63% by early 2026 [9].

The easy–hard gap is striking: the best individual system solves ~80% of easy tasks (<15 min) but only ~20% of hard ones (>1 h) [5]. Combined across all top systems, 95% of easy instances have been resolved by at least one agent, versus only 42% of hard ones—and among hard multi-file issues, only 9 of 25 have ever been solved. This ceiling effect, combined with contamination risk (half the issues predate 2020 and the repositories are widely used as training data [4]), raises concerns about the benchmark’s continued discriminative power. SWE-bench Pro, a harder leaderboard with novel uncontaminated problems, shows Claude Opus 4.5 dropping to ~46% [1].

Beyond SWE-bench. SWE-bench covers only Python bug-fixing in 12 repositories. Newer benchmarks begin to address broader scopes: SWE-bench Multilingual (300 tasks, 9 languages), CodeAgent-Bench (interactive development environments), and real-world deployment metrics. A comprehensive discussion of the evaluation landscape is deferred to §7.2.

4 Planning and Search Strategies for Code Synthesis

Zooming into the first critical stage: how agents decide *what* to generate before or during generation—decomposing problems, exploring solution spaces, and selecting among candidates.

4.1 Plan Representations and Decomposition

Pseudocode plans, hierarchical task decomposition, chain-of-thought outlines, dependency graphs, and specification sketches used to guide generation.

4.2 Search and Exploration Strategies

Tree/graph search over partial programs (MCTS, beam search, best-first), sampling-and-ranking, evolutionary approaches, and backtracking mechanisms.

4.3 Plan–Generate Interplay

How planning interleaves with code generation: plan-then-generate vs. interleaved planning, plan revision upon execution feedback, and plan grounding.

5 Multi-Agent Frameworks for Code Generation

An alternative architectural approach to the generation-to-validation span: multiple specialized agents divide labor by role (architect, coder, tester, reviewer), collectively covering planning, generation, and internal verification to improve code quality beyond what a single agent achieves.

5.1 Role Decomposition and Communication

How tasks are split among agents, what roles exist, and how agents exchange information (shared blackboard, message passing, structured dialogue).

5.2 Framework Designs

Survey of concrete frameworks (e.g., ChatDev, MetaGPT, AutoGen, CrewAI) and their orchestration mechanisms.

5.3 When Does Multi-Agency Help?

Conditions under which multi-agent designs outperform monolithic agents; coordination overhead, failure modes, and diminishing returns with agent count.

6 Execution Feedback and Iterative Refinement

Closing the loop: agents execute generated code, observe runtime outcomes (test failures, compiler errors, exceptions), and revise their output. Covers both general refinement and fault-specific self-repair.

6.1 Feedback Signals and Environments

Types of feedback: test results, compiler errors, runtime exceptions, static analysis, human approval signals. Sandboxed execution environments and their design.

6.2 Iterative Refinement Mechanisms

How agents incorporate feedback: reflexion-style verbal self-critique, repair prompts, diff-based editing, and multi-turn conversation with the environment.

6.3 Fault Localization and Self-Repair

When refinement becomes debugging: stack-trace analysis, LLM-driven root-cause reasoning, edit-based patching, candidate ranking, and test-driven repair loops.

6.4 Convergence and Stopping Criteria

When to stop iterating: pass-rate thresholds, diminishing improvement detection, budget limits, and the risk of over-fitting to weak test suites.

7 Discussion

7.1 Consolidated Patterns and Trends

Recurring design patterns across all five topics (observe-think-act loops, tool orchestration, LLM-as-judge verification). Macro trends: from single-shot prompting to agentic scaffolds, from monolithic agents to composable pipelines, from flat CoT to tree-structured search.

7.2 The Evaluation Landscape

Cross-cutting view of benchmarks (HumanEval, MBPP, SWE-bench, Defects4J, CrossCodeEval), their coverage of pipeline stages, saturation concerns, and gaps in realism.

7.3 The Context Window Bottleneck

How finite context constrains every pipeline stage; retrieval-augmented generation, hierarchical summarization, and repository-level chunking as mitigations.

7.4 Open Challenges

Unsolved problems: long-horizon reasoning, specification ambiguity, cost efficiency, trustworthiness, and human-in-the-loop control design.

7.5 Implications for Software Engineering Practice

How these advances may reshape developer workflows, team structures, and the economics of software production.

8 Conclusion

Summarize key findings across the five research fronts, restate the pipeline framing as a unifying lens, and point to the most promising future directions.

References

- [1] AgentMarketCap. SWE-bench verified: How AI coding agents went from 1.96% to 80.9% in 30 months. <https://agentmarketcap.ai/blog/2026/04/09/swe-bench-verified-progress-timeline-2023-2026>, 2026. Accessed: 2025-06-02.
- [2] Cognition AI. Introducing devin, the first AI software engineer. <https://cognition.ai/blog/introducing-devin>, 2024.
- [3] dataku. The SWE-bench verified leaderboard: Who's actually solving real bugs? <https://dataku.ai/blog/swe-bench-verified-leaderboard-who-solving-real-bugs>, 2025. Accessed: 2025-05-28.
- [4] Epoch AI. What skills does SWE-bench verified evaluate? <https://epoch.ai/publications/what-skills-does-swe-bench-verified-evaluate>, 2025. Accessed: 2025-06-01.
- [5] Jatin Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? <https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html>, 2025. Blog post, accessed: 2025-06-01.

- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.
- [7] OpenAI. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Accessed: 2025-06-01.
- [8] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. <https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model>, 2025. Accessed: 2026-05-28.
- [9] various. Dissecting the SWE-bench leaderboards: Profiling submitters and architectures of LLM- and agent-based repair systems. <https://arxiv.org/abs/2506.17208>, 2025. arXiv:2506.17208.
- [10] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.
- [11] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. ICLR 2025.
- [12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [13] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly?, 2025.
- [14] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.
- [16] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.